Complexity and Maintainability

1. Overview

A great system design is not the one with the most services.

A great design is:

- Simple enough to understand
- Easy to modify
- Easy to debug
- Easy to scale
- Reliable over time
- Low operational burden

This is where **Complexity** and **Maintainability** become critical non-functional requirements.

2. Definitions

A. Complexity

Complexity means how difficult the system is to:

- Understand
- Build
- Operate
- Scale
- Debug
- Change safely

More components + more dependencies + more rules = more complexity.

B. Maintainability

Maintainability means how easily engineers can:

- Fix bugs
- Add features
- Upgrade systems
- Replace components
- Onboard new developers
- Operate system long-term

3. Golden Rule

> **Do not design for requirements that do not exist.**

First step to reduce complexity:

```
None
Clarify functional requirements

Clarify non-functional requirements
```

If no need for global scale, don’t design global multi-region system.

If no need for real-time analytics, don’t add streaming stack.

4. Overengineering Is Common Mistake

Bad interview answer:

```
None
Microservices

Kafka

Kubernetes

20 databases

Event sourcing

Multi-region active-active
```

For a startup todo app.

That adds unnecessary complexity.

5. Prefer Simplicity First

Use:

- Monolith initially
- Single DB
- Simple cache
- Background jobs only if needed

Then evolve when scale demands it.

6. Separate Independent Components

As you draw HLD diagrams, identify components that can be isolated.

Examples:

- Auth service
- Notification service
- Search service
- Payments service
- Analytics pipeline

Benefits:

- Independent scaling
- Easier ownership
- Cleaner code boundaries

7. Reuse Common Shared Services

Instead of rebuilding same logic in every app, use shared infrastructure.

Common Reusable Services

A. Load Balancer

Routes traffic across instances.

B. Rate Limiter

Protects APIs.

C. Authentication / Authorization

Login, tokens, permissions.

D. Logging / Monitoring / Alerting

Visibility into production.

E. TLS Termination

SSL offload at ingress/load balancer.

F. Caching Layer

Shared Redis/CDN.

G. CI/CD Platform

Automated deploys.

8. Organization-Specific Shared Services

Larger companies may also share:

- Analytics platform
- Feature flag service
- ML inference platform
- User profile service
- Notification platform
- Audit platform

9. Complexity vs Reliability Trade-off

Higher availability often increases complexity.

Example:

Simple

```
1 region

1 DB

manual failover
```

Complex

None

```
multi-region active-active

cross-region replication

global load balancing

automatic failover
```

Need to justify complexity.

10. Complexity vs Performance Trade-off

Low latency systems may need:

- Cache layers
- Precomputation
- Denormalization
- Binary RPC
- Multiple replicas

These improve speed but add maintenance cost.

11. Complexity vs Cost Trade-off

More services =

- More infra cost
- More engineers
- More monitoring
- More on-call burden

12. Use Async Where Real-Time Not Needed

If task doesn't need immediate processing:

Use background jobs / ETL.

Examples:

- Reports
- Analytics
- Emails
- Thumbnail generation

Instead of synchronous request path.

This simplifies scaling.

13. Message Size Optimization

Network overhead matters in distributed systems.

Use compact communication formats.

REST + JSON

Readable, easy, larger payloads.

RPC + Binary Formats

Smaller and faster.

Examples:

- Apache Avro
- Apache Thrift
- Protocol Buffers

Trade-off:

Need schema management.

14. Metadata Services

Instead of embedding heavy configs everywhere:

Use metadata/config service.

Examples:

- Routing rules
- Feature flags
- Schema versions
- Tenant configs

Improves maintainability.

15. Operational Maintainability

A system is not maintainable if no one can operate it.

Need:

- Logs
- Metrics
- Alerts
- Dashboards
- Runbooks
- Backups
- Safe deploy process

16. Runbooks

Runbook = documented operational procedure.

Example:

```
None
DB high CPU:

1. Check slow queries

2. Scale replicas

3. Restart only if needed

4. Rollback recent deploy
```

Reduces outage chaos.

17. Failure Planning

Ask:

```
None
What if DB fails?

What if cache fails?

What if region fails?

What if queue backlog grows?
```

Then define mitigation:

- Replication
- Failover
- Retries
- Circuit breakers
- Degradation mode

18. Maintainable Code Architecture

Use:

- Clear module boundaries
- Interfaces
- Versioned APIs
- Good naming
- Tests
- Documentation

Avoid:

- Tight coupling
- Shared mutable state
- Hidden dependencies
- God services

19. Monolith vs Microservices

Aspect	Monolith	Microservices
Initial complexity	Low	High

Deployment simplicity	High	Lower
Team autonomy	Lower	Higher
Scale independently	Harder	Easier
Operational overhead	Low	High

Choose based on stage and team size.

20. Signals of Bad Complexity

Too many services for small product
No owner for components
Frequent outages after changes
Difficult onboarding
Hard local development
Unknown dependencies
Every change needs many teams

21. Interview Questions to Ask

1. Current scale or future scale?
2. Team size?
3. Time to market priority?
4. Compliance needs?
5. Need multi-region now?
6. Expected growth timeline?

22. Interview Answer Example

Q: How would you keep the design maintainable?

Answer:

> I'd start with the simplest architecture that meets requirements, reuse common infrastructure like auth/logging/caching, isolate domains with clear ownership, automate CI/CD, add observability, and evolve into separate services only when scale or team boundaries justify it.

23. Best Practices

- Start simple
- Avoid premature microservices
- Standardize shared components
- Document APIs
- Monitor everything important
- Automate deploys/tests
- Prefer stateless services
- Review architecture periodically

24. Trade-off Table

Decision	Simpler Now	Better Later
Monolith	Yes	Maybe limited later
Microservices	No	Scales teams later
Single DB	Yes	Simpler ops
Polyglot storage	No	Better specialization
Manual ops	Yes initially	Risk later
Automation	Setup cost	Long-term gain

25. Short Revision Notes

None

```
Complexity = Hardness to build/run/change


Maintainability = Ease of fixing and evolving


Reduce complexity:

- Clarify requirements

- Reuse shared services

- Keep architecture simple

- Use async jobs when possible

- Add observability

- Document runbooks
```